

MINI PROJECT — TECHNICAL PRESENTATION

ConferenceHub

Academic Events Management Platform

Domain:	Full-Stack Web Development
Frontend:	React.js (SPA)
Backend:	Node.js + Express.js
Database:	MongoDB (Mongoose ODM)
Real-time:	Socket.IO
Auth:	JWT + TOTP 2FA
Date:	February 2026

Project Overview

ConferenceHub is a full-stack, role-based Academic Events Management System designed to digitize the complete lifecycle of college conferences — from creation and registration, to live Q&A, QR-based attendance marking, certificate generation, and performance tracking via a leaderboard.

It replaces manual, paper-based event management with a real-time, secure, and responsive web application accessible across all devices.

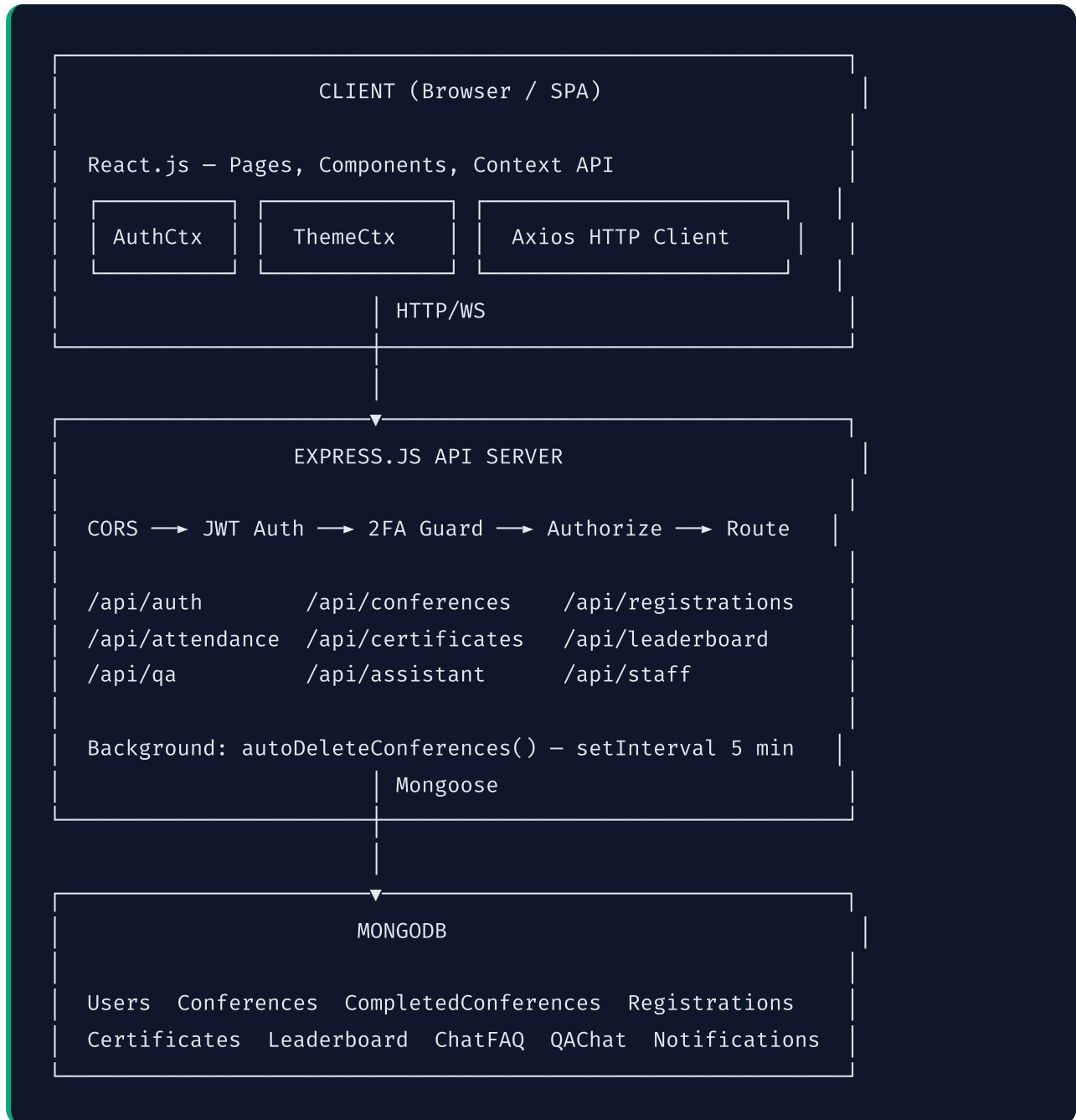
Problem Statement: College conferences typically require manual registration sheets, individual attendance calls, and offline certificate distribution — all of which are error-prone and time-consuming. ConferenceHub eliminates these inefficiencies through a unified digital platform.

Technology Stack

Layer	Technology	Purpose
Frontend	React.js (CRA, SPA)	Component-based UI with client-side routing
Styling	Vanilla CSS + CSS Variables	Theme-aware, responsive design system
Backend	Node.js + Express.js	REST API server, middleware pipeline
Database	MongoDB + Mongoose ODM	Document-based NoSQL data store
Real-time	Socket.IO	Bidirectional event-based communication (Live Q&A)
Authentication	JWT (JSON Web Tokens)	Stateless, role-encoded session management
2FA	Speakeasy (TOTP)	Google Authenticator-compatible OTP
QR Generation	qrcode.react	Generates JSON-encoded QR tickets client-side
QR Scanning	jsQR (CDN)	Decodes QR from canvas video frames at 100ms interval
Email	Nodemailer / Brevo API	Registration confirmation, OTP delivery
File Upload	Multer	Conference poster and certificate PDF handling

System Architecture

The application follows a classic **three-tier architecture** — Presentation, Logic, and Data — with an additional real-time layer powered by WebSockets.



Key Design Choice: The frontend communicates *exclusively* through the REST API — it never touches the database directly. This separation enforces security and keeps business logic in one place.

Role-Based Access Control (RBAC)

Every API route passes through a layered middleware chain before executing any business logic:

```
Request → protect() → authorize('staff','admin') → require2FAComplete() → Control
```

● Student

- › Register for conferences
- › View dept-filtered meetings
- › Download QR ticket
- › Download certificate
- › Earn leaderboard points
- › Ask KatBot questions

● Staff

- › All student capabilities
- › Create / delete conferences
- › Scan student QR codes
- › Upload certificates
- › Manage Q&A sessions
- › View completed conferences

● Admin

- › All staff capabilities
- › System-wide conference access
- › Leaderboard management
- › Audit log access
- › User management

Middleware	What It Does
protect()	Verifies JWT signature and expiry; attaches decoded user to req.user
authorize(...roles)	Checks req.user.role against allowed roles; returns 403 if unauthorized
require2FAComplete	Blocks access if user has 2FA enabled but hasn't verified the OTP this session

Key Feature Deep-Dives

5.1 — QR Code Attendance System

The QR attendance system uses a two-part mechanism: a **client-side QR generator** for students, and a **real-time camera-based QR scanner** for staff.

1 Registration

Student registers → backend generates unique ticket: `TICKET-{timestamp}-{random5}`

2 QR Generation (Client-side)

QRCodePage encodes a JSON payload into an SVG QR matrix via `qrcode.react`

```
{ registrationId, ticketNumber, userId, studentName, department }
```

3 Scanning (Staff)

Staff opens QR Scanner → camera stream captured by `<video>` → `jsQR` decodes frames from canvas every 100ms

4 Attendance API

`POST /api/attendance/mark` — validates ticketNumber against DB, marks status → `'attended'`, logs timestamp

5 Points Award

Backend awards leaderboard points via `calculatePoints('attend')` — skipped if the scanner is the conference creator

Why embed name/dept in QR? Embedding student details in the QR payload lets the scanner display a rich confirmation card (name, dept, ticket, timestamp) instantly — without any additional API call.

5.2 — Conference Lifecycle & Auto-Archival

Conference status is **never stored in the database** — it is computed dynamically on every read using `deriveConferenceStatus()`:

```
now < date          → "upcoming"  
date ≤ now ≤ endDate → "ongoing"
```

```
now > endDate      → "completed"
explicit cancellation → "cancelled"
```

A background job runs every 5 minutes via `setInterval()` :

```
autoDeleteConferences()
  1. Find non-cancelled conferences with endDate + 30min < now
  2. Archive to CompletedConference collection (preserves history)
  3. Check alreadyArchived to prevent duplicates
  4. Delete from live Conference collection (keeps hot queries fast)
  5. Staff who created the conference can view it under "My Completed Conference"
```

5.3 — Department-Based Meeting Visibility

Students only see conferences relevant to their department. This is enforced on the frontend using `useMemo` :

```
const visibleConferences = useMemo(() => {
  let filtered = conferences.filter(c =>
    ['ongoing', 'upcoming'].includes(c.status)
  );
  if (!canManageEvents && user?.department) {
    filtered = filtered.filter(c =>
      c.department === user.department // own dept
      || c.department === 'ALL'        // open to everyone
    );
  }
  return filtered;
}, [conferences, canManageEvents, user]);
```

5.4 — Two-Factor Authentication (TOTP)

1

Setup

Backend calls `speakeasy.generateSecret()` → returns base32 secret + `otpauth://` URI → frontend renders QR for Google Authenticator

2

Verification

User submits 6-digit TOTP → `speakeasy.totp.verify()` with 1-window tolerance → `twoFactorEnabled = true` stored in DB

3

Every Login

After password check → if 2FA enabled → user must verify OTP → JWT then issued with `twoFactorVerified: true` claim

5.5 — KatBot Virtual Assistant

An in-app chatbot that uses **keyword extraction + cosine-like overlap scoring** against a MongoDB FAQ collection:

```
Query: "How do I join a live meeting?"  
→ Stop-words removed: [ "join", "live", "meeting" ]  
→ Each ChatFAQ scored:  score = matching_keywords / total_query_keywords  
→ Best match returned  → hitCount++ in MongoDB  
→ Popular questions surface as suggestion chips
```

Why not an external LLM API? For a college intranet tool, a local keyword-matching approach keeps the system self-contained, offline-capable, fast (<50ms), and free. The `ChatFAQ` collection is extensible — new answers can be added without code changes.

Database Schema Design

Collection	Key Fields	Design Decision
User	name, email, password (bcrypt), role, department, twoFactorEnabled, twoFactorSecret	Password field uses <code>select: false</code> — never returned in queries by default
Conference	title, date, endDate, speaker{}, department, maxAttendees, createdBy → User	Status is derived at runtime — never stored. Prevents stale data.
CompletedConference	originalId, title, date, endDate, department, createdBy → User	Separate collection for historical data. Keeps live Conference queries fast.
Registration	userId, conferenceId, ticketNumber, status, qrCodeScanned, attendanceTime	Normalized — references both User and Conference independently
Certificate	userId, conferenceId, certificateNumber, filePath, downloadCount	downloadCount tracked for audit purposes
Leaderboard	userId, totalPoints, conferenceAttended, pointsHistory[]	History array allows per-conference point audit
ChatFAQ	question, answer, keywords[], category, hitCount, source	hitCount enables popularity-based suggestion ranking

Schema Relationships



User — Leaderboard (pointsHistory references Conference)
User — QChat (messages in a conference room)

References over Embedding: Mongoose references (`ObjectId ref`) are used instead of embedded sub-documents. This allows independent queries — e.g., fetching all registrations for a conference without loading user documents, or fetching all user registrations without loading full conference data.

Frontend Architecture

Layer	File / Module	Responsibility
Global State	AuthContext.js	User session, JWT, role flags (isAdmin, isStaff, canManageEvents)
Global State	ThemeContext.js	Dark/light mode persisted via localStorage
API Layer	utils/api.js	Axios instance with baseURL, auth headers, error interceptors
Helpers	utils/helpers.js	formatDate, formatTime, generateTicketNumber, calculatePoints
Page	StaffDashboard.js	Ongoing/Upcoming/Completed conferences, forms, Q&A, attendance stats
Page	Meetings.js	Department-filtered live meeting feed with registration-gated join
Page	ParticipationDashboard.js	Student's registration history, attendance stats, certificate access
Page	QRCodePage.js	Renders and downloads the student's QR attendance ticket
Page	QRScanner.js	jsQR-powered camera scanner; rich scan result display
Component	VirtualAssistant.js	KatBot floating chat widget (bottom-right, all pages)

Why React Context instead of Redux? The app's shared state is limited to authentication and theme — both shallow, rarely-updated values. Context API handles this without the boilerplate overhead of reducers, actions, and slices required by Redux.

Security Measures

PASSWORD SECURITY

bcrypt hashing with 10 salt rounds. Passwords never stored in plaintext. `select: false` on schema prevents accidental exposure in API responses.

JWT AUTHENTICATION

Short-lived tokens signed with HS256. Token contains user ID, role, and 2FA verification flag. Verified on every protected request.

TWO-FACTOR AUTH (TOTP)

TOTP tokens valid for ± 30 s window. Backup codes stored as bcrypt hashes. Sensitive routes blocked if 2FA not verified in current session.

ROLE-BASED MIDDLEWARE

Every route explicitly declares required roles. A student JWT cannot reach staff endpoints — authorization is checked server-side, never client-side only.

QR TICKET FORGERY PREVENTION

Server validates ticketNumber against the database independently — modifying the QR JSON and re-scanning is rejected if IDs don't match the stored registration.

FILE UPLOAD SAFETY

Multer restricts MIME types and file sizes. Uploaded files stored outside the public web root. Filenames sanitized to prevent directory traversal.

EMAIL CONFIRMATION

New accounts require email verification before login. Password reset uses time-limited tokens sent to the registered email address.

CORS POLICY

Server explicitly allows only the frontend origin via `CORS_ORIGIN` environment variable. All other origins are rejected at the network level.

Real-Time Communication (Socket.IO)

Socket.IO runs on the same HTTP server as Express, enabling full-duplex communication without polling:

Staff creates Q&A session for conferenceId

- Frontend: `socket.emit('join-room', conferenceId)`
- All participants join the same socket room

Student asks question:

- `socket.emit('qa:question', { conferenceId, question, userId })`
- Server: `io.to(conferenceId).emit('qa:new-question', questionData)`
- All participants receive instantly – no page refresh needed

Staff answers:

- `socket.emit('qa:answer', { questionId, answer })`
- Server: `io.to(conferenceId).emit('qa:answered', { questionId, answer })`
- All clients update answer in real-time

Why WebSockets over polling? HTTP polling sends a request every N seconds regardless of new data — wasteful and slow. WebSockets maintain a persistent connection; the server pushes events only when they occur, giving true real-time latency (<50ms).

Responsive Design System

Technique	Implementation
CSS Variables	All colors, shadows, and spacing defined as <code>:root</code> variables. Dark/light mode switches the entire palette by toggling a single <code>data-theme</code> attribute on <code><html></code>
Flexbox Layouts	Conference cards and dashboard rows use <code>display: flex; flex-wrap: wrap</code> — items reflow naturally without JavaScript
CSS Grid	Statistics grids and card collections use <code>grid-template-columns: repeat(auto-fill, minmax(280px, 1fr))</code> — columns auto-collapse on narrow screens
Media Queries	Breakpoints at 768px and 480px — multi-column becomes single-column, horizontal cards become vertical stacks
No CSS Framework	Vanilla CSS ensures sub-millisecond style resolution with zero dependency overhead

Project Summary — Concepts Demonstrated

CS Concept	Where Applied
Full-Stack Development	React ↔ Express ↔ MongoDB three-tier architecture
RESTful API Design	12 route groups with proper HTTP verbs, status codes, and JSON responses
Database Normalization	Reference-based Mongoose schema; no redundant data embedding
Authentication & Security	bcrypt, JWT, TOTP-2FA, CORS, session middleware chain
Role-Based Access Control	3-tier RBAC enforced server-side across all 40+ API routes
Real-Time Systems	Socket.IO WebSocket rooms for live Q&A and notifications
Background Processing	setInterval-based auto-archival job running server-side every 5 minutes
QR Technology	JSON-encoded QR generation + jsQR frame-by-frame video decoding
State Management	React Context API — auth state, theme state, per-page local state
Responsive UI/UX	CSS Variables + Flexbox + Grid + Media Queries — no CSS framework
Lightweight NLP	Keyword extraction + overlap scoring for chatbot FAQ matching
File Handling	Multer middleware for poster/certificate PDF upload with MIME validation
Email Integration	Transactional email via Nodemailer/Brevo for OTP and confirmation
Environment Configuration	.env for all secrets — DB URI, JWT secret, email credentials, CORS origin

